



Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ШАБЛони «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF
RESPONSIBILITY», «PROTOTYPE» »**
Музичний програвач

Виконав:
Студент групи ІА-23
Волошин В. В.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:.....	3
Хід роботи.....	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми.....	3
2. Реалізувати один з розглянутих шаблонів за обраною темою.....	4
Висновки:.....	6
Код проекту:.....	6

Тема:

ШАБЛони «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE»

Мета:

Ознайомитися з основними шаблонами проектування, такими як «Adapter», «Builder», «Command», «Chain of Responsibility», «Prototype» , дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Хід роботи

1. Реалізувати не менше 3-х класів відповідно до обраної теми

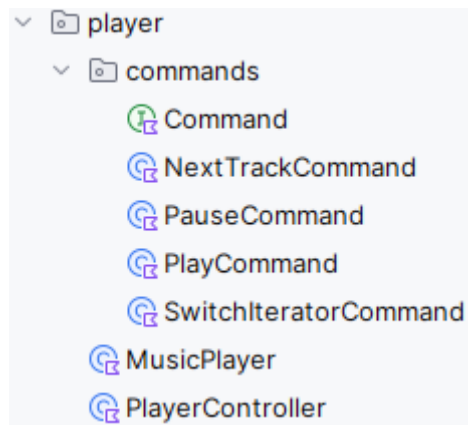


Рис. 1 — Структура проекту

У процесі виконання лабораторної роботи було створено сім класів, які забезпечують реалізацію функціоналу музичного програвача (рис. 1). Наведемо детальний опис кожного з цих класів:

1. MusicPlayer

Цей клас відповідає за основну логіку програвання музики. Він зберігає плейлист і підтримує ітератор для навігації по піснях у ньому. Клас дозволяє відтворювати пісні, ставити на паузу, а також переходити до наступного треку.

2. Command

Це абстрактний клас, який задає загальний інтерфейс для всіх команд. Він містить абстрактний метод execute(), який буде реалізований у підкласах.

3. PlayCommand

Цей клас реалізує команду для відтворення поточної пісні. При виклику методу `execute()` викликається метод `play()` класу `MusicPlayer`, який розпочинає відтворення поточної пісні.

4. PauseCommand

Клас, який реалізує команду для паузи відтворення пісні. Коли викликається метод `execute()`, він звертається до методу `pause()` класу `MusicPlayer`, зупиняючи програвання.

5. NextTrackCommand

Клас, який реалізує команду для переходу до наступного треку в плейлісті. При виклику методу `execute()` цей клас викликає метод `nextTrack()` у класі `MusicPlayer`, що дозволяє відтворити наступну пісню, залежно від типу активного ітератора.

6. SwitchIteratorCommand

Цей клас реалізує команду для зміни ітератора у `MusicPlayer`. Метод `execute()` дозволяє змінювати тип ітератора (наприклад, на послідовний, випадковий або повторюваний). Він використовує метод `setIterator()` класу `MusicPlayer`, щоб замінити поточний ітератор на новий.

7. PlayerController

Цей клас виконує роль інтерфейсу між користувачем і системою команд. Він відповідає за налаштування команд для кожної кнопки (наприклад, кнопки "Play", "Pause", "Next") та за їх виконання при натисканні на відповідну кнопку. Клас зберігає словник команд і викликає їх через метод `pressButton()`.

2. Реалізувати один з розглянутих шаблонів за обраною темою

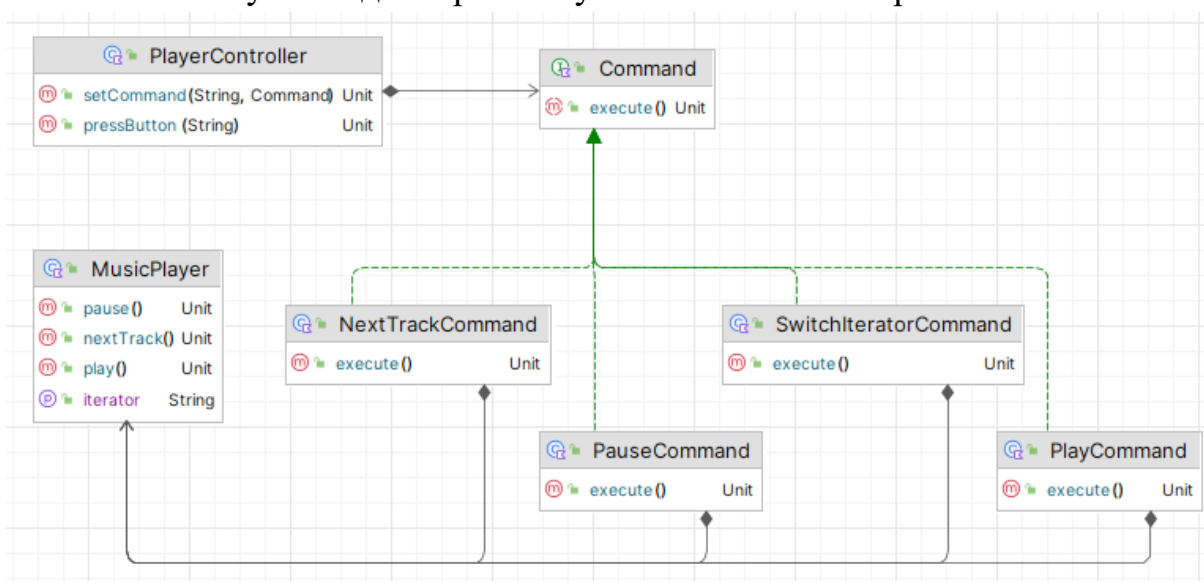


Рис. 2 — Діаграма класів

Патерн Command був реалізований у проєкті для того, щоб інкапсулювати запити користувача (наприклад, відтворення музики, пауза, перемикання пісень) в окремі об'єкти команд. Ці команди дозволяють програвачу виконувати конкретні дії без необхідності знати про деталі реалізації кожної з операцій.

Реалізація

- Команди: Для кожної дії користувача створено окремий клас команди (наприклад, PlayCommand, PauseCommand, NextTrackCommand, SwitchIteratorCommand), які інкапсулюють відповідні операції, що мають бути виконані в класі MusicPlayer.
- Інвокер: Клас PlayerController виступає як інвокер, який взаємодіє з користувачем, приймає натискання кнопок та викликає відповідні команди. Це дозволяє динамічно змінювати команди, які виконуються, без змін у самій логіці програвача.
- Приймач: Клас MusicPlayer є приймачем, який безпосередньо виконує дії за допомогою методів, таких як play(), pause(), nextTrack(), setIterator() тощо. Він отримує запити від команд та реалізує необхідні операції.

Проблеми, які вирішує патерн

- Інкапсуляція запитів: Кожна команда інкапсулює конкретний запит, що дозволяє легко додавати нові дії (наприклад, нові команди для роботи з музикою) без змін в інших частинах програми.
- Розширюваність: Легко додавати нові команди або змінювати вже існуючі, просто створивши нові класи команд.
- Гнучкість у виконанні операцій: Патерн дозволяє визначити та виконувати операції в будь-якому порядку. Наприклад, можна змінювати ітератори, не змінюючи логіку відтворення музики, і змінювати команду виконання на ходу.
- Управління діями: Клас PlayerController дозволяє гнучко керувати діями, асоціюючи їх з кнопками або іншими елементами інтерфейсу, без необхідності знати про реалізацію цих дій.

Переваги використання

- Гнучкість та модульність: Дії (команди) можна легко додавати, змінювати або видаляти без необхідності змінювати основну логіку програвача або інтерфейсу користувача.

- Керування запитами: Клас `PlayerController` дозволяє централізовано керувати всіма запитами користувача, надаючи можливість зберігати історію виконаних команд (якщо буде реалізовано в майбутньому).
- Зниження зв'язності: Клас `MusicPlayer` не знає, які конкретно команди його викликають, що робить систему більш гнучкою та легко масштабованою.

Висновки:

У ході лабораторної роботи було реалізовано патерн `Command` в музичному програвачі, що дозволило інкапсулювати операції управління програвачем у вигляді окремих команд. Це забезпечило гнучкість і модульність коду, дозволяючи легко додавати нові функції та змінювати існуючі без значних змін у системі.

Код проекту:

<https://github.com/vv0loshyn/TRPZ-labs>